Project: Serverless & Microservice Patterns

Due: 02/28/2026 11:59 PM
Extended Deadline: 03/01/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

knative
- Task Overview
- Task

grpc
- Task Overview
- Task

dapr
- Task Overview
- Task

Wrap Up
- Conclusion
- End of project survey

grpc (Task)

After scaling up your LLMServer using FastAPI, your boss goes to a conference on LLM Serving, and comes back with a new proposition - changing your FastAPI server to a new system: gRPC. As it's used at Google, and uses their own homemade solution for serialization, he expects that this will result in potentially even faster code and potentially even better code in the long run.

You sit down to write a simple gRPC server in Python, and add your LLM code to it to assess the performance accordingly.

A Brief Introduction to gRPC

gRPC is another synchronous communication system that was originally designed by Google to handle their own distributed systems workload.

The name itself used to stand for "Google Remote Procedure Calls", but nowadays the project has diverged from the original Google implementation into it's own fairly stable and reliable system.

Here, a **Remote Procedure Call** is a specific type of API and system rooted in distributed computing, which heavily influences Cloud Native architectures. Unlike traditional messaging systems, RPC provides a different abstraction for communication, allowing functions to be executed remotely as if they were local.

In past projects, you wrote web servers that basically **accepted** and **served** requests. You were responsible for:

1. **Input Validation:** Making sure that the user did not pass in strange data or additional data.
2. **Request and Response Handling:** Making sure that the parameters you need are there or not.

In FastAPI, we accomplished this through the use of **types**, but as you now know, adding static typing to any project, even a fairly "simple" one, takes some effort, and you still need to ensure that your client respects it.

These issues are endemic to a request-response system with the **interface** you have - you need to worry about what to send and how to send it, as well as the validation stage. It's not pretty, but it's useful to do.

However, if you think about it carefully, you'll realize that a lot of what we have done seems a bit silly. After all, you know, as well as I do, that our system is literally just **trying to run a function on another machine and send the output back**. As we design our systems to be stateless, we effectively just have a **function** that we want to get some outputs of, that we just want to run on another computer instead of the client's computer.

This simple idea is known in distributed computing as a **remote procedure call**. You have some server that provides a **function** to a client, and the client can invoke it through the network. gRPC handles the validation, typing, and other related semantics for you, while you focus on writing code.

gRPC separates the **service code** from the **application code**. You write your application-specific code, **a couple of specification files for the input and output message**, and then the library will generate **stubs**, which serve to handle the details of the client architecture for you.

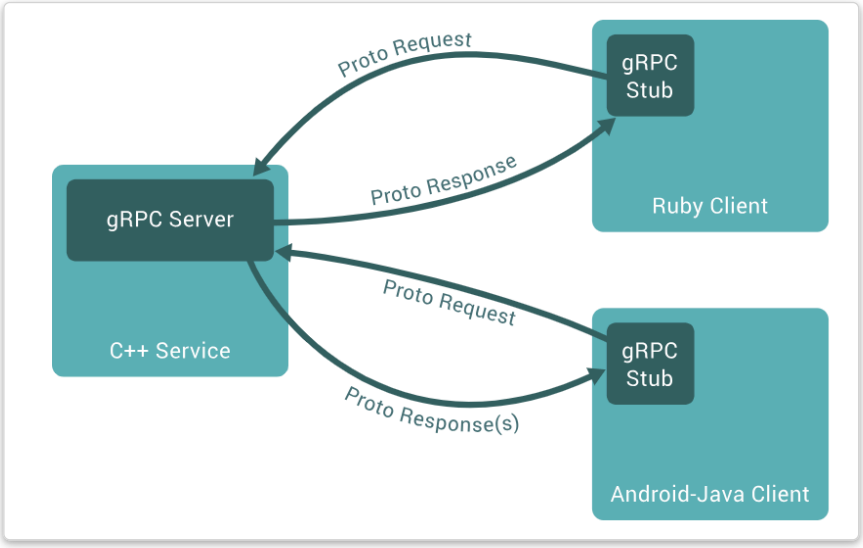


Figure 1: The gRPC architecture, taken from https://grpc.io/docs/what-is-grpc/introduction/

As you can see from the above diagram, this allows you to use **separate client and server code for communication**, which allows us to make microservices as opposed to just separate Python threads or processes.

In order to allow for this interop, gRPC abstracts away **messaging** as much as possible. Many gRPC implementations support client-side load balancing via built-in name resolution. Furthermore, specific gRPC libraries provide the infrastructure to handle concurrent requests across multiple worker threads or processes to maximize throughput. The only really tricky part of the system is **the communication system** gRPC uses by default - **protocol buffers**

Protocol Buffers - What and Why

As a system, gRPC by default uses **protocol buffers** to both validate the data and handle it accordingly. A **protocol buffer** is another project by Google that allows you to **turn requests and responses into binary data**.

This is primarily useful as it allows you to **reduce the amount of data you send over the network**. Our LLM service is slow mostly due to the compute we are giving it, but in most systems one of the primary limiting factors is actually **network bandwidth**.

In the past, we have mostly sent messages as strings. To approximately measure how much data we send on each response of our LLMService, we will use **sys.getsizeof**, which gets the number of bytes of a Python object:

Code:

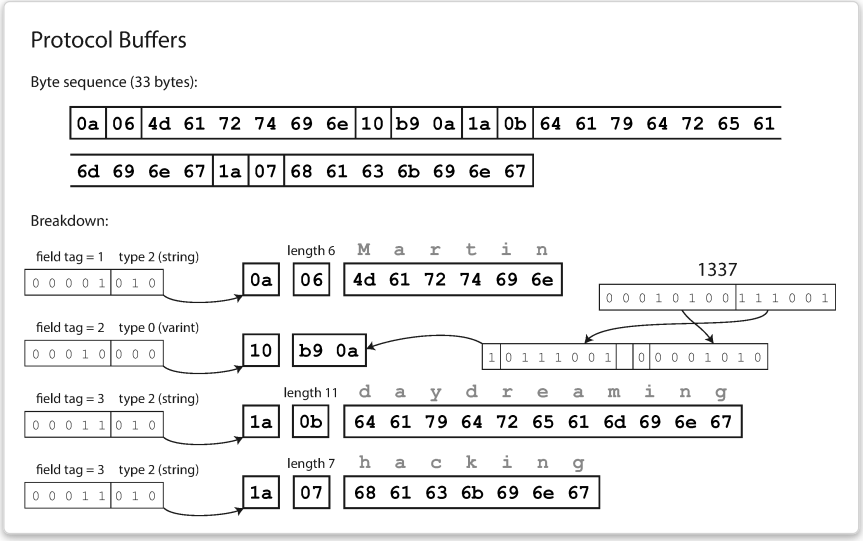
```
import sys

output_message = """{
  "id": "cmpl-186c31c2-9803-4421-a9aa-81db9ab4ea30",
  "object": "text_completion",
  "created": 1708482398,
  "model": "tinyllama-1.1b-chat-v1.0.Q2_K.gguf",
  "choices": [
    {
      "text": "\n     A Taco is a dish consisting of a corn tortilla filled with refried
      "index": 0,
      "logprobs": null,
      "finish_reason": "length"
    }
  ],
  "usage": {
```

Run

Output:

As you can see, this message takes about 800 bytes to send. gRPC, on the other hand, **will convert your message to a binary format and send the information directly**.

If you see this diagram, from Martin Kleppman's book "Designing Data-Intensive Applications", you can see an example of what they are doing:



I know this looks really complex, but all they are doing is the following:

1. They are converting each key into a **tag**, which is just a number from 1 to the number of tags.
2. Instead of **encoding each element as a string**, they send the native datatype, along with a **length** so that the parser knows how much data they sent.

This approach saves a ton of bytes, especially if you are sending **numeric** datatypes like logits or arrays.

The following sandbox is an example of what protobuf does under the hood, taking the fields from your JSON and encoding them as a single set of bytes and using **struct** to pack the JSON as a simple binary file instead.

Before you run the code, guess how many bytes you think this might be. The answer may surprise you.

Information

We are **not** expecting you to understand this code in any depth. If you do want to know more about struct, and the details of what this code is doing, see the excellent docs.. If you would like something that goes into the depths of binarizing strings and the complexities that come there, check out chapter 4 of **Fluent Python, 2nd Edition**.

Code:

```
import struct
import json

# Sample JSON message
output_message = {
  "id": "cmpl-186c31c2-9803-4421-a9aa-81db9ab4ea30",
  "object": "text_completion",
  "created": 1708482398,
  "model": "tinyllama-1.1b-chat-v1.0.Q2_K.gguf",
  "choices": [
    {
      "text": "\n     A Taco is a dish consisting of a corn tortilla filled with refried
      "index": 0,
      "logprobs": "null",
      "finish_reason": "length"
    }
```

Run

Output:

As you can see, we have about **200** bytes in our packed message, instead of our initial **800**. Even for a message with mostly strings, **serialization can have tremendous benefits for reducing network latency**.

The flip-side is that **serializing data is hard**. You effectively need to write a **parser** to handle the bytes, and a systematic way to **specify how to pack the bytes**. As we are using gRPC, this particular issue is less relevant, but there are some side-effects that you should know:

> You will need to **respect the intent of the designers of Protobuf**. They focused on objects that are only **a few megabytes in size each**.

You will be unable **to compare two objects just by comparing the bytes**. Protobufs are deterministic, but the libraries change heavily from week to week using the same versions of the buffer and focus on field-value comparisons.
They are not the best for matrices and other ML-style objects. The protobuf developers recommend FITS for this purpose.
They are not available in all languages, but in particular scientific languages like FORTRAN.

Information

It is easy to think of FORTRAN as an old language that no one uses, but it is still used by the scientific community in certain fields. This is mostly due to the existence of maintainers, but this is an important thing to consider if you have a legacy codebase you want to port.

In JSON, printing the object allowed you to read it. Procol buffers **need the specification in order for them to make any sense**.
Protocol buffers are not a formal standard of any organization, so they are unsuitable if there is a legal requirement to use **standardized systems**.

While protocol buffers have their drawbacks, their pros are worth learning about, as they have a really nice interface that leads well to the kinds of tasks you are likely to work on with LLMs and other AI-enabled systems. They are a common inference choice in TensorFlow, and while PyTorch users prefer REST, it is not too hard to make PyTorch use gRPC as well.

Information

We will be unable to cover everything about protocol buffers and serialization in this course. It's a large field in its own right, and this project is close to the maximum amount of work we think is viable for a student.

If you want to learn everything, just read Chapter 4 of **Designing Data-Intensive Applications** by Martin Kleppman. The book is huge, but it's one of the best written books on scaling an application that we've ever seen.

Here are a few points that are good-to-knows that we will not cover in this project:

1. You **can** use JSON and gRPC together, instead of using a protocol buffer. As protobufs are one of the core reasons to use gRPC, it's more for legacy applications, but still usable. See this section of the documuentation for more..
2. While it's not intended to be used with the Web, **you can use gRPC on the web with gRPC-web**. This blogpost by the creators of gRPC goes into more detail, but we would just like to add that, for the most part, **keeping client-side APIs human-readable** is usually a priority, as it helps outside developers understand your API better.

With this brief introduction to what a protocol buffer is (it's a fancy binary version of an object) and what it's used for (to specify requests and responses in gRPC), let's start re-writng our LLM Service into gRPC.

In general, the steps for doing this migration follows a three phase process:

1. Write the **.proto** file that specifies your protocol buffer schema.
2. Generate the stubs and Python client information.
3. Migrate your server logic and test with a Python client.

Let's start with SubTask1, and write our first ProtoBuf specification.,

SubTask 1: Your First ProtoBuf Specification

Like in previous projects, we will make use of Poetry to try to create dependency isolation and dependency locking. This is especially important for **protocol buffers**, as you have to **precisely match the library version between the server and client for anything to work**. In your hardout directory run the following command:

```
cd project3
```

```
poetry new grpc-p4
```

Running the above command will generate a `src` directory. Note that we will not implement anything in this directory. All modifications will be made outside the `src` directory.

Information

Again, if you want to use a virtual environment that is within the package folder like we did, remember to create a **poetry.toml** with the following:

```
[virtualenvs]
in-project = true
path = ".venv"
```

It's not required to pass the graders, but it might make your life a bit easier.

Then, let's install all of the needed packages for gRPC for this build:

Go to the grpc-p4 directory by running **cd grpc-p4** and run the following commands:

```
poetry add grpcio-tools
poetry add llama-cpp-python
```

As you can see, we really are only installing the following:

1. **llama-cpp-python**, the library that allows us to perform LLM inference.
2. **grpcio-tools**, which installs both **gRPC** and **the protobuf compiler**

This is all we will need for gRPC. As it is not a **REST** framework, no need to worry about **WSGI vs ASGI** or even **what Python web server to host it on after we write the code**. Instead, gRPC has you focus more on just **the abstraction** and wants to take some of that burden away from you.

The only thing we need to write for this subtask is our protocol buffer specification.

```
syntax = "proto3";

package test.latest;

service HelloService {

rpc SayHello (HelloRequest) returns (HelloResponse);
}

message HelloRequest {
  string greeting = 1;
}

message HelloResponse {
  string reply = 1;
}
```

While we have not introduced you all to how ProtocolBuffer and gRPC schema files work, you can probably make out the following:

1. We define a gRPC service named **HelloService**, with a particular function of interest called **SayHello**.
2. It takes in a **HelloRequest** object, that contains the string **greeting**.
3. It returns a **HelloResponse** object, that contains the string **reply**.

Does it make some sense? After all, we will need to specify **what message we want to send and where**, as well as **what capabilities our server shall provide to our client**.

What might seem a bit strange is the syntax. Let's go over some of the hairier parts step by step.

1. The first line `syntax="proto3";` is our **version syntax specifier**. ProtocolBuffer specifications have had multiple **versions** come out as the years progress. It's best to consider each of these are **fairly different**, closer to the differences between Python 2 and Python 3 than small updates to a larger system. As a result, when you write a protocol buffer file, **you should always specify the version**. If you do not, your **protocol buffer compiler and stub generator would fail**. More recently, these have been replaced by yearly **edition** markers, but we will focus on Protobuf 3 as it is most widespread.
2. The second line `package test.latest;` defines a **specific namespace** for the protocol buffers for this service. This is like the **namespaces** you have worked with in Kubernetes - they provide a way for you to make sure that there are no **namespace conflicts**, and keep **teams separated**.
3. Here the **= 1** part simply refers to the **serialization order** we want our objects to be put in. Each **item in a message object** (including sub-objects) should have **their own field number. We generally give incrementing field numbers to each item in a message object.**. Here is an example with three fields and a nested sub-field so you understand what we are getting at:

```
message ExampleNest {

   message LittleBird {
      string call = 1;
      string color = 2;
   }

   LittleBird myfirstbird = 1;
   LittleBird mysecondbird = 2;
   string NestName = 3;
}
```

Here, we have 2 LittleBird messages that are inside of a larger ExampleNest message. The **LittleBird** messages each get their own **call** and **color** field, and we would access them like a Python class object and its fields. This nesting is quite similar to your Pydantic validation system seen in earlier projects, so keep it in mind as you work on this task.

> **Information**
>
> These field numbers directly relate to how **your data will get encoded by gRPC**. While you have **no control** how that happens, **you cannot reuse field numbers**. You just cannot without specifying a new system, or without making it clear to old consumers that they are using a **different version of your API**.
>
> Once you finalize the number, you **must keep it to preserve interoperability**. **Protobuf** is not aware of the names of each field, and uses the field number to understand **what and where each item is**. You need to **maintain consistency or else you can fail**.

It will be up to you to figure out exactly how you want to tackle specifying each and every type for your buffer - we will just prescribe the types you will need to pass the autograder, and how we shall verify that you did it correctly. In general, we will try to match functionality as close as possible to our LLM server user in Task 1.

Create a folder called **llmproto** inside the **outer grpc-p4** folder, and then add a file called **llm.proto** inside of a **v1** folder within **llmproto**. This will serve as your gRPC spec file for this task. You must use the **llmproto.v1** package.

In it, you must create three separate sections:

1. Your service specification. The service **must** be called `LLMService`, the function it provides must be called **Completion**, and it must take in a **CompletionRequest** message object and return a **CompletionResponse** message object.
2. Your CompletionRequest **message specification**. This should be fairly straightforward, only containing the **message** field, which is a `string`.

3. To make your life easier, we have kept the types for the **CompletionResponse** message object relatively simple:
   a. **Each and every numeric response should be an** `int32` **object.**
   b. **Each and every string response should be a** `string` **object.**
   c. The only exception is **created**, which should use the **Timestamp** type. In addition to the base types shown in the documentation, protobufs contain a ton of **useful extension types** that you can use. You will need to import the Timestamp type from `google/protobuf/timestamp.proto`, after which you will be able to use it.

Building and managing stubs

Now, you can try running the compiler to try to generate your stubs and server code. Make sure you're in the `grpc-p4` directory before running the command below:

```
poetry run python -m grpc_tools.protoc -Igrpc_p4=. --python_out=. --p
```

This will create three new files:

grpc_p4/llmproto/v1/llm_pb2_grpc.py, which contains the **autogenerated** LLMService **interface/stub**.
grpc_p4/llmproto/v1/llm_pb2.py, which contains a glimpse of the real serialization code.
grpc_p4/llmproto/v1/llm_pb2.pyi, which contains the protocol-buffer message **interface files**.

Take about 15 minutes to **try** to read this code, and then follow along in the next section as we break it down and get you started on building a gRPC server and client!

The following is a high level discussion of the **pyi** type. You do not need to read it to complete the project.

Information

At this point, you are probably wondering what an **Python interface file** is, and why this is the first time you are hearing of it. After all, if Python has other file formats than its **.py** format, why are you just hearing about it now?

The main reason, like most good reasons, is that these files **are more advanced**, and are attempts at adding **some new programming methodologies** to Python that require us to dig into weeds for a bit. They also are not necessarily that popular, as they do not always come up when discussing best practices.

A template file, in short, **defines the interface to something, but not the implementation**. If you had an array class, for example, the **interface** would be the operations it supports, whereas the **implementation** would be **how** the class actually performs them.

They originate from languages like C and C++, where they are called **header files**, and are used to make it easier for people **to pick up what someone else is trying to provide**. Their main benefit is that, if you use them diligently, it becomes very easy for someone else to use your system, as **you can add deeper documentation there**, and provide **type hints** to give your users something more to work off of than just the names of the fields to the function you wrote.

The trade-off you make is that **you need to plan your project before writing it**. This can be tricky, as you are tasked with telling people **how** you want the tool to work **before you write it**, which can lead to you missing important inputs and outputs. Moreover, once you write it, as **Python** will not check it by itself, you will need to **maintain this additional file**.

For the purposes of this project, this header file will be **verified by gRPC**, and **automatically generated by it**. As a result, you really only need to write the protobuf schema and the actual class to do anything - everything else is just there to make it easier to pick up your code afterwards.

For the purposes of your career, however, things get a bit murkier. In general, there are two different schools of thought when it comes to documenting code:

1. **Code must be as simple as possible, and "self-documenting"**. It must be so obvious and so clear that it does not need **any additional writing**.
2. **Code must be well-documented**. Everything must have some information on it, and it needs to be precise enough that users can make sense of it and turn their goals into reality.

Python **interface** files fall into the latter. They can make it **much easier** to see the various inputs and outputs of a large project, and are usually perfect **when the code starts to get a bit too much out of hand**. They however **tend to add a lot of additional work to your plate**. One of

the instructors once spent a hundred hours documenting and testing a multimodal machine learning library, and it is not being heavily used at the moment.

Moreover, they are used for **mypy** to perform static type checking on your system, and separate **types** from the actual Python code implementing the types. This can make your code a bit less busy, and also make it easier to separate API design from API implementation.

Subtask 2: Writing Your First gRPC Client and Server

At this point, you have a protocol buffer file, a few automatically generated files, and are probably wondering how to begin writing this server. In project 2, the tasks of learning the framework and developing the system were mostly left to you. In this task, we will lead you through the task.

If you want to, you can skip this section and build the gRPC client and server using the documentation's quickstart. The rest of this subtask is a more in-depth run through of the files and everything else going on, so you can make the most of this system.

Breaking down gRPC's auto-generated files

As it is fairly new, gRPC with version 3 protocol buffers lacks a fair bit of examples for our purposes. The docs, in this case, **are not completely accurate about the implementation**. Therefore, let's go through the sample **hello-world** example that gRPC provides in their github repository..

For our purposes, we shall focus on first understanding the **autogenerated files**, as that is what you have, and then **the basic client and server**. Let's start with the **pb2.py** file:. As we will use the line numbers to refer to different sections, we encourage you to open the github link we provide to make it easer. We provide the text of the files for completeness sake, and in case the files update.

helloworld_pb2.py

At a glance, this file looks really hard to read. We recommend that you copy the below code into your favourite text editor. The instructions thereafter makes reference to line numbers which you cannot see from this minimal code block interface.

```
# -*- coding: utf-8 -*-
# Generated by the protocol buffer compiler.  DO NOT EDIT!
# source: helloworld.proto
# Protobuf Python Version: 4.25.0
"""Generated protocol buffer code."""
from google.protobuf import descriptor as _descriptor
from google.protobuf import descriptor_pool as _descriptor_pool
from google.protobuf import symbol_database as _symbol_database
from google.protobuf.internal import builder as _builder
# @@protoc_insertion_point(imports)

_sym_db = _symbol_database.Default()




DESCRIPTOR = _descriptor_pool.Default().AddSerializedFile(b'\n\x10hel

_globals = globals()
_builder.BuildMessageAndEnumDescriptors(DESCRIPTOR, _globals)
_builder.BuildTopDescriptorsAndMessages(DESCRIPTOR, 'helloworld_pb2',
if _descriptor._USE_C_DESCRIPTORS == False:
  _globals['DESCRIPTOR']._options = None
  _globals['DESCRIPTOR']._serialized_options = b'\n\033io.grpc.exampl
  _globals['_HELLOREQUEST']._serialized_start=32
  _globals['_HELLOREQUEST']._serialized_end=60
  _globals['_HELLOREPLY']._serialized_start=62
  _globals['_HELLOREPLY']._serialized_end=91
  _globals['_GREETER']._serialized_start=94
```

```
  _globals['_GREETER']._serialized_end=322
# @@protoc_insertion_point(module_scope)
```

We will only concern ourselves with a few particular sections here, as most of it is meant to be interface materials for gRPC internals. **Whenever you see an underscore (_) in front of a variable name, think of it as a pointer to "something internal to Python or a language"**.

On **line 2**, you can see that this was generated by the protocol buffer compiler, and that we **should not edit the file**. For this subtask, you will need to **make sure you do not edit anything on these files**. When you have auto-generated content, you **should never edit the files** as it is usually really easy to replace them and break your code as a result. While there are systems that you can use to then automatically edit these snippets, I would recommend **just learning how to work with them manually**.

On **line 19**, there is a call to **globals**. This is a dictionary of **global variables** for the program, and it is what gRPC uses internally to make sure that anything that accesses this file **suddenly has access to the underlying variables needed for gRPC**. Anytime you have to work with **C/C++** code directly, you generally need to have syntax like this.

On **lines 23-30**, we have our variable declarations for our global variables. All we want you to see is the following::

1. There are fields for **_HELLOREQUEST** and **_HELLOREPLY**. Each of these define the **messages for each protocol buffer we created**.
2. There is also a field for our **service**, which in this case is **_GREETER**. Again, you should check that your **pb2.py** file has this, as otherwise imports may not work correctly.
3. On each of these lines, you can see that we set **serialization_start** and **serialization_end** variables. This is what **protobuf** uses to make sure that it accesses the right data.

All this file does, then, is **define the protocol buffer message types**. Think of it as where you go to to specifically **import a message** and then **create a message**.

Let's move on to the other **py** file.

helloworld_pb2_grpc.py

At a glance, this looks substantially better:

```
# Generated by the gRPC Python protocol compiler plugin. DO NOT EDIT!
"""Client and server classes corresponding to protobuf-defined servic
import grpc

import helloworld_pb2 as helloworld__pb2


class GreeterStub(object):
    """The greeting service definition.
    """

    def __init__(self, channel):
        """Constructor.

        Args:
            channel: A grpc.Channel.
        """
        self.SayHello = channel.unary_unary(
                '/helloworld.Greeter/SayHello',
                request_serializer=helloworld__pb2.HelloRequest.Seria
                response_deserializer=helloworld__pb2.HelloReply.From
                _registered_method=True)
        self.SayHelloStreamReply = channel.unary_stream(
                '/helloworld.Greeter/SayHelloStreamReply',
                request_serializer=helloworld__pb2.HelloRequest.Seria
```

```
                response_deserializer=helloworld__pb2.HelloReply.From
                _registered_method=True)
        self.SayHelloBidiStream = channel.stream_stream(
                '/helloworld.Greeter/SayHelloBidiStream',
                request_serializer=helloworld__pb2.HelloRequest.Seria
                response_deserializer=helloworld__pb2.HelloReply.From
                _registered_method=True)


class GreeterServicer(object):
    """The greeting service definition.
    """

    def SayHello(self, request, context):
        """Sends a greeting
        """
        context.set_code(grpc.StatusCode.UNIMPLEMENTED)
        context.set_details('Method not implemented!')
        raise NotImplementedError('Method not implemented!')

    def SayHelloStreamReply(self, request, context):
        """Missing associated documentation comment in .proto file."'
        context.set_code(grpc.StatusCode.UNIMPLEMENTED)
        context.set_details('Method not implemented!')
        raise NotImplementedError('Method not implemented!')

    def SayHelloBidiStream(self, request_iterator, context):
        """Missing associated documentation comment in .proto file."'
        context.set_code(grpc.StatusCode.UNIMPLEMENTED)
        context.set_details('Method not implemented!')
        raise NotImplementedError('Method not implemented!')


def add_GreeterServicer_to_server(servicer, server):
    rpc_method_handlers = {
            'SayHello': grpc.unary_unary_rpc_method_handler(
                    servicer.SayHello,
                    request_deserializer=helloworld__pb2.HelloRequest
                    response_serializer=helloworld__pb2.HelloReply.Se
            ),
            'SayHelloStreamReply': grpc.unary_stream_rpc_method_handl
                    servicer.SayHelloStreamReply,
                    request_deserializer=helloworld__pb2.HelloRequest
                    response_serializer=helloworld__pb2.HelloReply.Se
            ),
            'SayHelloBidiStream': grpc.stream_stream_rpc_method_handl
                    servicer.SayHelloBidiStream,
                    request_deserializer=helloworld__pb2.HelloRequest
                    response_serializer=helloworld__pb2.HelloReply.Se
            ),
    }
    generic_handler = grpc.method_handlers_generic_handler(
            'helloworld.Greeter', rpc_method_handlers)
    server.add_generic_rpc_handlers((generic_handler,))


 # This class is part of an EXPERIMENTAL API.
class Greeter(object):
    """The greeting service definition.
    """
```

```
    @staticmethod
    def SayHello(request,
            target,
            options=(),
            channel_credentials=None,
            call_credentials=None,
            insecure=False,
            compression=None,
            wait_for_ready=None,
            timeout=None,
            metadata=None):
        return grpc.experimental.unary_unary(
            request,
            target,
            '/helloworld.Greeter/SayHello',
            helloworld__pb2.HelloRequest.SerializeToString,
            helloworld__pb2.HelloReply.FromString,
            options,
            channel_credentials,
            insecure,
            call_credentials,
            compression,
            wait_for_ready,
            timeout,
            metadata,
            _registered_method=True)

    @staticmethod
    def SayHelloStreamReply(request,
            target,
            options=(),
            channel_credentials=None,
            call_credentials=None,
            insecure=False,
            compression=None,
            wait_for_ready=None,
            timeout=None,
            metadata=None):
        return grpc.experimental.unary_stream(
            request,
            target,
            '/helloworld.Greeter/SayHelloStreamReply',
            helloworld__pb2.HelloRequest.SerializeToString,
            helloworld__pb2.HelloReply.FromString,
            options,
            channel_credentials,
            insecure,
            call_credentials,
            compression,
            wait_for_ready,
            timeout,
            metadata,
            _registered_method=True)

    @staticmethod
    def SayHelloBidiStream(request_iterator,
            target,
            options=(),
            channel_credentials=None,
            call_credentials=None,
            insecure=False,
```

```
            compression=None,
            wait_for_ready=None,
            timeout=None,
            metadata=None):
        return grpc.experimental.stream_stream(
            request_iterator,
            target,
            '/helloworld.Greeter/SayHelloBidiStream',
            helloworld__pb2.HelloRequest.SerializeToString,
            helloworld__pb2.HelloReply.FromString,
            options,
            channel_credentials,
            insecure,
            call_credentials,
            compression,
            wait_for_ready,
            timeout,
            metadata,
            _registered_method=True)
```

Walking through it section by section:

Lines 8-32 define the `GreeterStub` class. This is part of the **interface** that the server provides to the client, and it defines the specific **functions** you will need to implement. As we can see, this project does not just contain 1 service but **3**, a **unary** system that supports a single request and response, a **streaming reply** service that supports a stream of outputs for your message, and a **bidirectional stream** when you need to send and reply with streams of information.

Lines 35-56 define the `GreeterServicer` class, which is what you will **subclass** to implement your gRPC service. Think of the **stub** as a **guide for what exists**, whereas the **Servicer** is the actual interaction system.

Lines 59-79 define the **add_GreeterServicer_to_server** function. As we briefly discussed in the last project, some microservice frameworks require you to **create a server object** and then **add methods to it**. This method is what you will use to **link your functions to a running server**.

Lines 83-166 define a new **experimental API** for defining and registering services to your **gRPC** server. We will not touch these for our implementation, and suggest you ignore this for now.

Lastly, let's cover the **pyi** file:

helloworld_pb2.pyi

At a glance, this file is also **less useful** for our purposes:

```
from google.protobuf import descriptor as _descriptor
from google.protobuf import message as _message
from typing import ClassVar as _ClassVar, Optional as _Optional

DESCRIPTOR: _descriptor.FileDescriptor

class HelloRequest(_message.Message):
    __slots__ = ("name",)
    NAME_FIELD_NUMBER: _ClassVar[int]
    name: str
    def __init__(self, name: _Optional[str] = ...) -> None: ...

class HelloReply(_message.Message):
    __slots__ = ("message",)
    MESSAGE_FIELD_NUMBER: _ClassVar[int]
```

```
    message: str
    def __init__(self, message: _Optional[str] = ...) -> None: ...
```

It re-defines the message types, and more importantly **defines the input and output for creating a message**. The **HelloRequest** and **HelloResponse** objects that are used here are **much simpler** than the ones you have, but they just show the **types** that are needed for them to work, and it allows **mypy** to provide type checking.

Now that we've finished our tour of the autogenerated files, let's go to the **example client and server files** that gRPC provides.

greeter_client.py

This file is much easier to read at a glance:

```
"""The Python implementation of the GRPC helloworld.Greeter client."'

from __future__ import print_function

import logging

import grpc
import helloworld_pb2
import helloworld_pb2_grpc


def run():
    # NOTE(gRPC Python Team): .close() is possible on a channel and s
    # used in circumstances in which the with statement does not fit
    # of the code.
    print("Will try to greet world ...")
    with grpc.insecure_channel("localhost:50051") as channel:
        stub = helloworld_pb2_grpc.GreeterStub(channel)
        response = stub.SayHello(helloworld_pb2.HelloRequest(name="yo
    print("Greeter client received: " + response.message)


if __name__ == "__main__":
    logging.basicConfig()
    run()
```

On a high-level, all this code does is the following:

1. Open the connection to the **gRPC server**, which handles the response.
2. Send a simple message with the word: "you" as a parameter. Here, we use the **stub** class, as that is what the client will see when it comes to the server code.
3. Get the response, and then print it out to you. Note that **gRPC requests and responses** are **objects**, not **dictionaries**.

Going through it line by line (using the line numbers off of GitHub):

Line 16 is something you can ignore, and honestly remove. gRPC's python interface was originally built for **Python 2**, and thus it imports this to get **Python 3** style prints.

Line 18 imports the logging module. Logging is effectively a **higher-powered** version of printing - we may cover this in a later project, but for now all this does is ensure that **gRPC errors** reach you when you are debugging.

Line 30 uses a **with** statement to create an **insecure channel**, i.e. something like an HTTPS channel. gRPC **comes with built-ins** for a wide variety of channel-specific encryption schemes, including SSL/HTTPS. They also have support for **ALTS-style** encryption, which is a modified version of **mTLS** that Google uses in production, but it's **only available on Google's Cloud System**.

Line 31 creates the stub, which we will use to **interact** with the channel and get responses.

Line 32 creates the **HelloRequest** object and sends it to the server. gRPC automatically creates the constructor and the arguments based on the message type.

Overall, it's quite similar to how someone may use a normal REST API. The only real difference is that, instead of dictionaries, you will need to use **objects**. If you **need** to make sure that something exists here while debugging, a simple way to check that some field exists in the object is to convert it to a dictionary and then get the keys using **a.dict.keys()**.

First Step and Interim Validation

You'll first implement the client portion of your gRPC project.

1. Ensure you're in the `grpc-p4` directory. Make sure not get lost here as all commands you will be running will be made in this directory.
2. Create a `grpc_client_standard.py` file in the `grpc_p4.py` folder. Take note of the underscore in the folder name. Don't confuse this with the grpc-p4 folder.
3. Copy and paste the following code into the file you just created:

```
"""The Python implementation of the GRPC helloworld.Greeter client

from __future__ import print_function

import logging

import grpc
from grpc_p4.llmproto.v1 import llm_pb2, llm_pb2_grpc



def run():
    # NOTE(gRPC Python Team): .close() is possible on a channel an
    # used in circumstances in which the with statement does not f
    # of the code.
    print("Will try to send a message ...")
    with grpc.insecure_channel("localhost:50051") as channel:
        # TODO: Instantiate your stub here


        # TODO: Make a request to the server here and save it in a


if __name__ == "__main__":
    logging.basicConfig()
    run()
```

4. To make the `llm_pb2` and `llm_pb2_grpc` imports work properly, your project needs to be installed as a Python package. Right now, Python doesn't automatically know that your project root should be part of its module search path.
   We fix this by:

   Updating the packages section of the `pyproject.toml` file to exactly this: `packages = [{ include = "grpc_p4", from = "." }]`. Here, we're telling poetry to look for packages in the project root.
   Installing the project in editable mode. Run the below command to do this. Don't forget the (dot) at the end of the command.

```
poetry run pip install -e .
```

1. In order to test your implementation, run the following command:

```
poetry run python grpc_p4/grpc_client_standard.py
```

While you will not be able to get it working without the server, you should get an error like this:

```
grpc._channel._InactiveRpcError: <_InactiveRpcError of RPC that termi
        status = StatusCode.UNAVAILABLE
        details = "failed to connect to all addresses; last error: UN
        debug_error_string = "UNKNOWN:Error received from peer  {grpc
```

This indicates that your code is functional, as we got to the point where gRPC just tried to send the message and it couldn't find the server.

Step 2 - Build your Server

With this knowledge, go and implement your server under `grpc_4/server.py`.

In theory, this only should involve the following:

Copying the sample server code into the file you want to run, say grpc_p4/server.py.
Copying over the code required to get the LLM to output something.
Changing the calling logic to create the message objects that we expect.

Here's a starter code you can use as reference. Note that the imports used for Fields in the CompletionResponse might be slightly different from your implementation. That doesn't mean you should change your .proto file. Rather, you should refactor the starter code to suit your .proto implementation.

```
from concurrent import futures
import logging

import grpc

from grpc_p4.llmproto.v1.llm_pb2_grpc import LLMServiceStub
from grpc_p4.llmproto.v1.llm_pb2 import CompletionResponse, Choice, U
from grpc_p4.llmproto.v1.llm_pb2_grpc import LLMServiceServicer
from grpc_p4.llmproto.v1.llm_pb2_grpc import add_LLMServiceServicer_t


from llama_cpp import Llama
from google.protobuf.timestamp_pb2 import Timestamp


def get_llm_response(message, llm):
    """
    Calls the LLM and returns a dict that looks like the llama_cpp co
    You're free to change this prompt format/parameters.
    """
    system_message = "You are a helpful assistant."
    prompt = message
    template = f"""<|system|>
    {system_message}</s>
    <|user|>
    {prompt}</s>
    <|assistant|>"""
    return llm(template, temperature=0.0, max_tokens=None)

class LLMService(LLMServiceServicer):
    """
    Implements the RPCs defined in your .proto:
    service LLMService { rpc Completion(CompletionRequest) returns (C
    """

    def Completion(self, request, context):
        """
```

```
        request: CompletionRequest with a 'message' field.
        returns: CompletionResponse matching your .proto specificatic
        """
        # TODO: invoke your lmm by passing in a message from the serv
        # store the llm response in a variable, say llm_response.

        # TODO: construct the CompletionResponse to send back to the
        # Note that this should match the specification of your .prot
        # The response from the llm is always structured the same, if
        # the response looks like, run your container image locally a
        # Based on that response, you can construct the sub-parts of

        # [HINT]: to get the choices dictionary, you can use the foll
        # choice_dict = llm_response["choices"][0]

        # To construct the choices part of the CompletionResponse, yo
        # choices = [
        #     Choice(
        #         text=choice_dict["text"],
        #         ..........
        #         ..........
        #         ..........
        #     )
        # ]

        # TODO: Based on the above example, you should complete the e


        # TODO: return a completion response with all specified fielc
        # return ..........
        pass


def serve():
    port = "50051"
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    add_LLMServiceServicer_to_server(LLMService(), server)
    server.add_insecure_port("[::]:" + port)
    server.start()
    print("Server started, listening on " + port)
    server.wait_for_termination()


if __name__ == "__main__":
    global llm
    # The model path is the location of your model file.
    # If you used a different path for the previous projects, update
    llm = Llama(model_path="../../llm_server/tinyllama-1.1b-chat-v1.(

    logging.basicConfig()
    serve()
```

Here are a couple of things that will make your life easier:

1. When you want to **create** your **created** Timestamp object, import **Timestamp** from **google.protobuf.timestamp_pb2**. You can then create a Timestamp object using **Timestamp().FromSeconds(...)**.

Test your application

To test your application, make sure the server is running before invoking `poetry run python grpc_p4/grpc_client_standard.py`.

A similar command is used to run the server, you only change the file name. You should open two terminals to do this.

Your subtask ends when **you are able to** send a request with your **client**, get the server to **respond**, and the response is what you expect, i.e. it has an object with the right type that our grader can understand.

If you think you're lost or taking too long to complete this, reach out for assistance.

> **Danger**
>
> You may run into an error that reads like the following:
>
> “instruction crashed due to server illegal instruction (core dumped)”
>
> A workaround is to run the following command: `CMAKE_ARGS="-DLLAMA_AVX2=OFF -DLLAMA_FMA=OFF -DLLAMA_F16C=OFF" FORCE_CMAKE=1 pip install llama-cpp-python --no-cache-dir --force-reinstall`

SubTask 3: Output Streaming

At this point, you have a working **client** and **server** application. However, something that **gRPC** offers that standard **REST** systems cannot easily provide is **request and response streaming**.

> **Information**
>
> Please move on if you do not want the gory explanation behind what **easily** means here. It's not important for this class, but may be useful for later work.
>
> Technically, **HTTP** systems support a kind of streaming known as **Server-Side Events**. In it, the client **requests for some output**, and the server **periodically sends a response** that the client then reads to get the output. In fact, this is **supported by FastAPI**, and is likely part of how **gRPC** actually does events.
>
> The main issue is that **handling** these events can be unintuitive and/or tricky, especially when you scale.
>
> With external clients, those outside of your cluster, SSE is **technically fine**, but it's not always supported **by your application load balancer**. Moreover, **your API gateway** might dislike it, as it is outside of the scope of general **external clients**. Externally, it's usually best to **expose** your system as some sort of **REST** API, while internally you can use **streaming responses** to make your life easier.
>
> gRPC is nice in that **streaming responses are fairly easy to consume and produce**. You do not need to worry as much about the details behind streaming, and can focus on just **getting it working**.

For our service, **streaming** is particularly worth doing as it allows us to **reduce the latency**. Part of the reason that our service takes so much time is that **it has to produce the entire response** to our client before sending. Luckily, **llama-cpp-python** can produce a stream of responses for you to send out, each containing a singular token that our LLM produces. This can **cut down the total latency**, and give users **an idea of how far along we are**. It also lets us **stop producing output** when we want to.

Before continuing, we have prepared an example notebook that contains an example of how to use **llama-cpp-python** with our model to stream outputs. It is literally a **one-line change**, but **run it** to make sure that you are aware of what the output should be.

One point that we want to talk a **tiny** bit about is the print statement. It uses the **end** and **flush** keyword arguments, which you may not have seen before:

> **end** tells print what to output when the print command finishes. In our case, each element of the stream prints out a **tiny portion** of the final text, so we need to **print that output without adding a newline**.
> **flush** tells Python to send the output **as soon as we send print**. By default, standard console output (stdout) does not always allow us to see the output of a print statement without someone flushing the **stdout** buffer. Without this line, our function **still gets the input**, but we will not see the streaming output.

To enable this functionality, and more faithfully replicate ChatGPT for our system, we will need to make a few changes to our system:

1. We will need to update our protocol buffer to take in and respond with **streaming** inputs and outputs.
2. We will need to update our **server** to ensure that it can support streaming.
3. We will need to **create a new client** to demonstrate streaming.

We will give some high-level guidance to help you on your way, but **most** of this involves copying, pasting, and modifying the code you have:

Step 1: Updating our protobuf definition.

Your objective here is to **update** the protocol buffer definition to include the **streaming response function** as well as **StreamingRequest** and **StreamingResponse** message types.

You must do the following:

For the **rpc** definition under **LLMService**, create a new function, give the **function** the name **LLMStream**. At the end you should have two functions in the `LLMService` section: one returns a `CompletionResponse` and the other `LLMStreamResponse`.
Make the input request type **LLMStreamRequest**, and give it the exact same parameters as the **CompletionRequest**. The reason why we are writing a separate request and response type is because **this gives us flexibility for changing the input and output types later on**.
Make the output request type **stream LLMStreamResponse**, and make it almost a copy of **CompletionResponse**, except for ommitting the **usage** field, which is not present in our **llama-cpp-python** service. Stream keyword tells gRPC that the response would be streamed.
Once these changes are completed, you have to recompile your .proto file to generate the stubs necessary for streaming. Use the below command to accomplish this:

```
poetry run python -m grpc_tools.protoc -Igrpc_p4=. --python_out=. --p
```

Step 2: Updating our service definition.

Now, we get to one of the main tasks - **actually getting the server to use a streamingresponse**.

You will need to do the following:

1. **Import** the new message request and response types.
2. **Modify** the LLM code you have to **allow it to output both a stream and a normal output**. You do this by adding a new function to the LLMService class. This function should be `LLMStream` and should take the same parameters as the `Completion` function, and should **return** an `LLMStreamResponse`.
3. **Write the actual streaming response**
4. When implementing server-side streaming, your RPC should return an iterator of responses instead of a single response. That means:

   Call your LLM in streaming mode so it produces partial outputs incrementally.
   Loop over each streamed chunk the LLM returns.
   For each chunk:
   - Extract the first item in choices
   - Build a Choice message from that chunk's `text`, `index`, and `finish_reason`
   - Convert the `created` field into a google.protobuf.Timestamp
   - yield a stream response message back to the client

Out of these four, the **final** step is the only thing new. You will need to write a **generator function** instead of a **normal function** to stream your output. As a reminder, this is a function that uses **yield** instead of **return** and allows a Python function to **return a stream of information**.

Here's an example generator function to help speed you along - it returns the numbers from 1 to 5:

Code:

```
def cool_stream():
    for i in range(5):
        yield i


for elem in cool_stream():
    print(elem)

```

Run

Output:

Before you continue, **check that you can start the server without error**, and that you can still **get the completionresponse with your client** There will be more errors to debug, but this is a simple check to make sure you **imported everything correctly**.

Step 3: Updating our client definition

For this step, you will want to create a new client file called **grpc_p4/grpc_client_streaming.py**.

We suggest starting with the code **you already wrote for the last client**, and just making a few modifications:

1. Import the necessary **LLMStreamRequest** object.
2. Your focus should only be on the `run` function. You'll use the same stub as the one used in the prior client.
3. Like the generator function example above, use something like **for elem in stub.LLMStream(...** to get the streaming response.
4. Use **print(elem.choices[0].text, end="", flush=True)** to print your output to console so that you can confirm that the **streaming** happens.

Once you have finished, **run the server in one terminal and the client in another**. You will be able to see the **output** stream fairly quickly to you, which will make it clear that **you are doing way more than just a simple request response API**.

Asynchronous gRPC code

Like with FastAPI LLM server in previous projects, we shall focus on **getting this system** to use **async** and **await**. While this does look really daunting, we will work through it step by step. Part of the reason we want you to do this is for **performance**. gRPC's example **server**, which we used to create our **synchronous gRPC code**, uses **futures**, which according to them **is not optimal for performance**.

As a result, we will upgrade our implementation to use **asyncio**, an asynchronous web server in **Python**. This will involve changing only a **handful** of lines, but first **install** it:

```
poetry add asyncio
```

Don't forget to import `asyncio` in both client and server files.

For your **server.py** file, do the following:

1. Remove the **from concurrent import futures** line.
2. Change **def** to **async def** for both **LLMStream** and **Completion**.
3. Change **server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))** to **server = grpc.aio.server()**.
4. Change **def** to **async def** for **serve()**.
5. Add **await** before **server.start()** and **server.wait_for_termination()**.
6. Change **serve** to **asyncio.run(serve())**

For your **client.py** file, do the following:

1. Add **async** before **def run()**.
2. Replace `with grpc.insecure_channel("localhost:50051") as channel:` with `async with grpc.aio.insecure_channel("localhost:50051") as channel:`
3. Add `async` before the **for** statement
4. Change **run** to **asyncio.run(run())**

As you can see, adding async to our system is fairly drop-in.

Pre-Submission Validation

At this point, check that your server and client both run, and that you can still get a response with asyncio.

Prepare to Submit your Work

Download grpcurl

Before submitting, please make sure that your gRPC server is running. During testing, we will use **grpcurl**, a curl variant that lets us ping your server with grpc. You can install it locally using:

```
curl -L -o grpcurl.deb https://github.com/fullstorydev/grpcurl/releas

sudo dpkg -i grpcurl.deb

rm grpcurl.deb
```

Or by looking at the other instructions..

Points to Note Before Submission

1. Make sure your **protocol buffer definition** is in the following location: `your-project-root/project3/grpc-p4/llmproto/v1/llm.proto`. If you followed our instructions properly, this shouldn't be a problem.
2. The submitter assumes your `server` is running. Make sure to start the server before making a submission.

3. To get full scores, make sure your `grpc_client_standard` and `grpc_client_streaming` implementations return successful responses and do not error out.
4. If your're getting the right responses and not getting full scores from the grader, reach out ASAP.

Tree Structure for this Task

```
grpc-p4
├── Makefile
├── README.md
├── docker
│   └── Dockerfile
├── grpc_p4
│   ├── __init__.py
│   ├── grpc_client_standard.py
│   ├── grpc_client_streaming.py
│   ├── llmproto
│   │   └── v1
│   │       ├── llm_pb2.py
│   │       ├── llm_pb2.pyi
│   │       └── llm_pb2_grpc.py
│   ├── server.py
├── llmproto
│   └── v1
│       └── llm.proto
├── poetry.lock
├── poetry.toml
├── pyproject.toml
├── src
│   └── grpc_p4
│       └── __init__.py
└── tests
    └── __init__.py
```

CD into your project root folder and make a submission by running:

```
./submitter task2
```

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform